

Name:S.Harshavardhan

Reg;192524106

Course:CSA1718-AI

Experiment 1: Map Coloring using Constraint Satisfaction Problem (CSP)

AIM

To implement the **Map Coloring Problem** using the **Constraint Satisfaction Problem (CSP)** approach in Python such that no two adjacent regions have the same color.

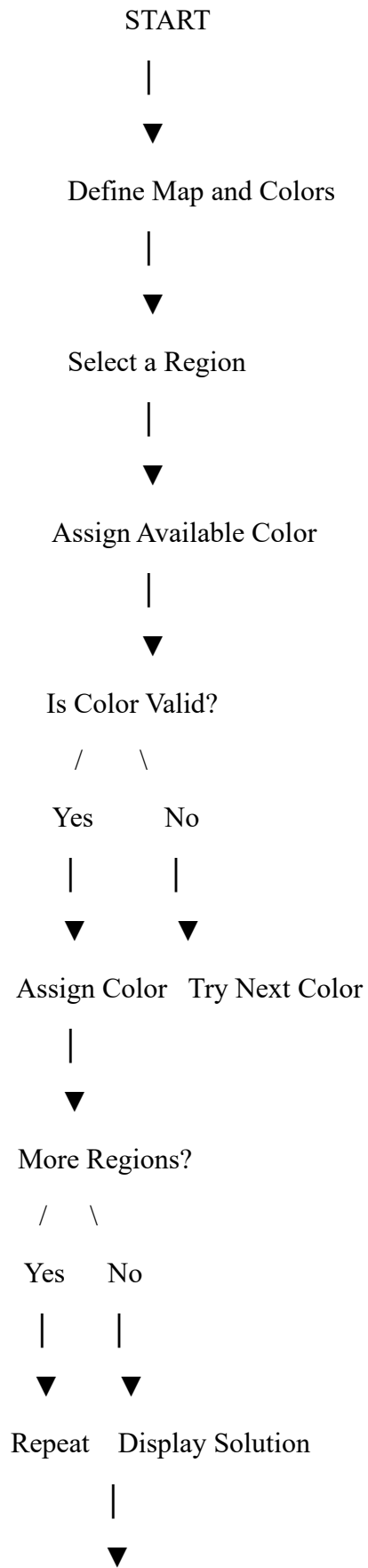
OBJECTIVE

- To understand the concept of Constraint Satisfaction Problems.
- To assign colors to regions while satisfying constraints.
- To ensure neighboring regions do not share the same color.

ALGORITHM

1. Start.
2. Define the map as a graph.
3. Define available colors (Red, Green, Blue).
4. Assign colors to each region one by one.
5. Check whether the selected color violates any constraint.
6. If valid, assign the color.
7. Otherwise, try another color.
8. Repeat until all regions are colored.
9. Display the final color assignment.
10. Stop.

FLOWCHART



STOP

PYTHON CODE

Map Coloring using CSP

```
states = ['A', 'B', 'C', 'D']
```

```
neighbors = {  
    'A': ['B', 'C'],  
    'B': ['A', 'C', 'D'],  
    'C': ['A', 'B', 'D'],  
    'D': ['B', 'C']  
}
```

```
colors = ['Red', 'Green', 'Blue']
```

```
solution = {}
```

```
def is_valid(state, color):  
    for neighbor in neighbors[state]:  
        if neighbor in solution and solution[neighbor] == color:  
            return False  
    return True
```

```
def solve(index):  
    if index == len(states):  
        return True
```

```
    state = states[index]
```

```
for color in colors:
    if is_valid(state, color):
        solution[state] = color
        if solve(index + 1):
            return True
        del solution[state]
```

```
return False
```

```
if solve(0):
    print("Map Coloring Solution:")
    for state in states:
        print(state, "->", solution[state])
else:
    print("No Solution Found")
```

SAMPLE OUTPUT

Map Coloring Solution:

A -> Red
B -> Green
C -> Blue
D -> Red

```
1 states = ['A', 'B', 'C', 'D']
2
3 neighbors = {
4     'A': ['B', 'C'],
5     'B': ['A', 'C', 'D'],
6     'C': ['A', 'B', 'D'],
7     'D': ['B', 'C']
8 }
9
10 colors = ['Red', 'Green', 'Blue']
11 solution = {}
12
13 def is_valid(state, color):
14     for neighbor in neighbors[state]:
15         if neighbor in solution and solution[neighbor] == color:
16             return False
17     return True
18
19 def solve(index):
20     if index == len(states):
21         return True
22
23     state = states[index]
24
25     for color in colors:
26         if is_valid(state, color):
27             solution[state] = color
28             if solve(index + 1):
29                 return True
30             del solution[state]
31
32     return False
33
34 if solve(0):
35     print("Map Coloring Solution:")
36     for state in states:
37         print(state, "->", solution[state])
38 else:
39     print("No Solution Found")
```

Enter Input here

If your code takes input, add it in the above box before

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.84 Mb

Your Output

Map Coloring Solution:
A -> Red
B -> Green
C -> Blue
D -> Red

RESULT

The Map Coloring Problem was successfully implemented using the Constraint Satisfaction Problem (CSP). The program assigned colors to all regions without violating the adjacency constraints.

CONCLUSION

The experiment demonstrated how CSP can solve the Map Coloring Problem efficiently using backtracking. The solution ensures that no two neighboring regions are assigned the same color, satisfying all constraints.

Experiment 2: Tic-Tac-Toe Game using Python

AIM

To develop a **Tic-Tac-Toe game** in Python where two players take turns placing **X** and **O** on a 3×3 board until one player wins or the game ends in a draw.

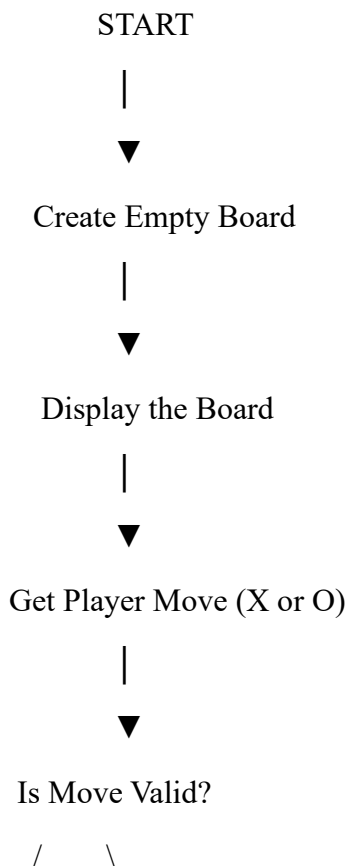
OBJECTIVE

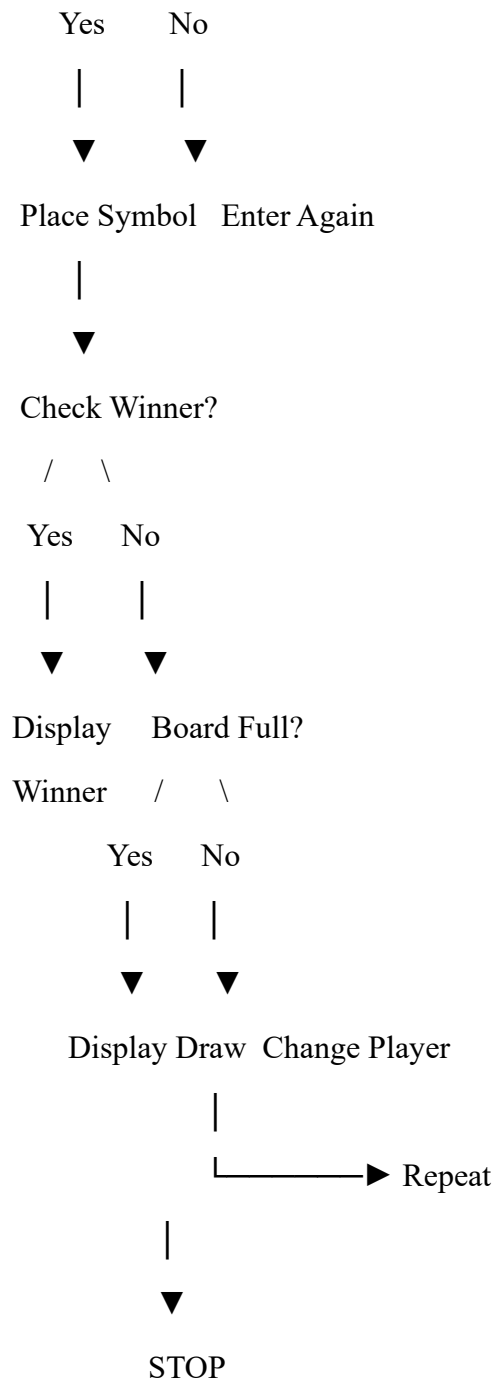
- To understand the implementation of a Tic-Tac-Toe game using Python.
- To learn the use of loops, functions, and conditional statements.
- To determine the winner or draw based on game rules.

ALGORITHM

1. Start.
2. Create a 3×3 Tic-Tac-Toe board.
3. Set the first player as **X**.
4. Display the board.
5. Ask the current player to enter row and column.
6. Check whether the selected position is empty.
7. If valid, place the player's symbol.
8. Check if the player has won.
9. If all cells are filled without a winner, declare a draw.
10. Otherwise, switch to the other player and continue.
11. Stop.

FLOWCHART





PYTHON CODE

Tic-Tac-Toe Game

```
board = [" " for _ in range(9)]
```

```
def print_board():
```

```
print()
print(board[0], "|", board[1], "|", board[2])
print("--+---+--")
print(board[3], "|", board[4], "|", board[5])
print("--+---+--")
print(board[6], "|", board[7], "|", board[8])
print()
```

```
def check_winner(player):
    wins = [
        [0,1,2],[3,4,5],[6,7,8],
        [0,3,6],[1,4,7],[2,5,8],
        [0,4,8],[2,4,6]
    ]
    for combo in wins:
        if all(board[i] == player for i in combo):
            return True
    return False
```

```
player = "X"
```

```
for turn in range(9):
    print_board()
    pos = int(input(f"Player {player}, enter position (1-9): ")) - 1

    if board[pos] == " ":
        board[pos] = player

    if check_winner(player):
```



```

    print_board()
    print(f'Player {player} Wins!')
    break

```

```

    player = "O" if player == "X" else "X"
else:
    print("Position already occupied!")
else:
    print_board()
    print("Game Draw!")

```

SAMPLE OUTPUT

```

| |
--+---+--

| |
--+---+--

| |

```

Player X, enter position (1-9): 1

Player O, enter position (1-9): 5

Player X, enter position (1-9): 2

Player O, enter position (1-9): 9

Player X, enter position (1-9): 3

```

X | X | X
--+---+--

| O |
--+---+--

| | O

```

Player X Wins!

```
board = [" " for _ in range(9)]

def print_board():
    print()
    print(board[0], "|", board[1], "|", board[2])
    print("-----")
    print(board[3], "|", board[4], "|", board[5])
    print("-----")
    print(board[6], "|", board[7], "|", board[8])
    print()

def check_winner(player):
    wins = [
        [0,1,2],[3,4,5],[6,7,8],
        [0,3,6],[1,4,7],[2,5,8],
        [0,4,8],[2,4,6]
    ]
    for combo in wins:
        if all(board[i] == player for i in combo):
            return True
    return False

player = "X"

for turn in range(9):
    print_board()
    pos = int(input(f"Player {player}, enter position (1-9): ")) - 1

    if board[pos] == " ":
        board[pos] = player

        if check_winner(player):
            print_board()
            print(f"Player {player} Wins!")
            break

        player = "O" if player == "X" else "X"
    else:
        print("Position already occupied!")
    else:
        print_board()
```

RESULT

The Tic-Tac-Toe game was successfully implemented in Python. The program allows two players to play alternately and correctly identifies the winner or declares a draw.

CONCLUSION

The experiment demonstrated the implementation of a Tic-Tac-Toe game using Python. It helped in understanding decision-making, loops, functions, and game logic, making it a simple application of Python programming.

Experiment 3: Minimax Algorithm for Gaming using Python

AIM

To implement the **Minimax Algorithm** in Python for selecting the optimal move in a two-player game.

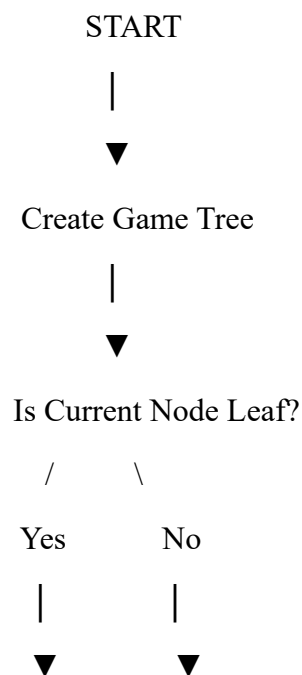
OBJECTIVE

- To understand the working of the Minimax algorithm.
- To implement game tree search using recursion.
- To determine the best move for the MAX player by considering the opponent's optimal moves.

ALGORITHM

1. Start.
2. Create a game tree with leaf node values.
3. Identify MAX and MIN levels.
4. If the current node is a leaf node, return its value.
5. If it is a MAX node, choose the maximum value among its children.
6. If it is a MIN node, choose the minimum value among its children.
7. Continue recursively until the root node is evaluated.
8. Display the optimal value at the root.
9. Stop.

FLOWCHART



Return Value MAX or MIN?

/ \

MAX

MIN

|

|



Choose Maximum Choose Minimum

|



Return Best Value

|



Display Result

|



STOP

PYTHON CODE

Minimax Algorithm

```
import math
```

```
def minimax(depth, nodeIndex, isMax, values, height):
```

```
    if depth == height:
```

```
        return values[nodeIndex]
```

```
    if isMax:
```

```
        return max(
```

```
            minimax(depth + 1, nodeIndex * 2, False, values, height),
```

```

        minimax(depth + 1, nodeIndex * 2 + 1, False, values, height)
    )
else:
    return min(
        minimax(depth + 1, nodeIndex * 2, True, values, height),
        minimax(depth + 1, nodeIndex * 2 + 1, True, values, height)
    )

# Leaf node values
values = [3, 5, 6, 9, 1, 2, 0, -1]

height = int(math.log2(len(values)))

print("Optimal Value:", minimax(0, 0, True, values, height))

```

SAMPLE OUTPUT

Optimal Value: 5

The screenshot shows a code editor with the following Python code:

```

# Minimax Algorithm
import math

def minimax(depth, nodeIndex, isMax, values, height):
    if depth == height:
        return values[nodeIndex]
    if isMax:
        return max(
            minimax(depth + 1, nodeIndex * 2, False, values, height),
            minimax(depth + 1, nodeIndex * 2 + 1, False, values, height)
        )
    else:
        return min(
            minimax(depth + 1, nodeIndex * 2, True, values, height),
            minimax(depth + 1, nodeIndex * 2 + 1, True, values, height)
        )

# Leaf node values
values = [3, 5, 6, 9, 1, 2, 0, -1]

height = int(math.log2(len(values)))

print("Optimal Value:", minimax(0, 0, True, values, height))

```

On the right side of the editor, there is a 'Run' button and a 'Visualize' button. Below them is an input field labeled 'Enter Input here' and a checkbox labeled 'If your code takes input, add it in'. The 'Output' section shows the following results:

Status : Successfully executed	
Time:	Memory
0.0100 secs	9.236 MB
Your Output	
Optimal Value: 5	

RESULT

The Minimax Algorithm was successfully implemented using Python. The program correctly evaluated the game tree and determined the optimal value for the MAX player.

CONCLUSION

The Minimax Algorithm efficiently selects the best possible move in a two-player game by assuming both players play optimally. It is widely used in AI-based games such as Tic-Tac-Toe and Chess for decision-making.